

Lossless Neural Compression

Pranav Sitaraman*, Gavin Ye*, and Alex Todoran*

Harvard University
Cambridge, MA 02138, USA,
{pranavsitaraman,gye,atodoran}@college.harvard.edu

Abstract. Learning based compression has long emerged to compete or even outperform classical lossy compression methods. However, few research has been done in *lossless* learning based image compression. Although prior research have shown that entropy-based large machine learning model can improve compression rates, these models are extremely computationally costly, and most can only be run on GPUs. We consider the problem of devising an efficient lossless compression algorithm that combines standard classical compression schemes, such as PNG, with a small machine learning model that is capable of exploiting more complex redundancies in images that is computationally efficient. Specifically, we develop a compression framework that stores images as the sum of a “block-coloring” quantized image, efficiently encoded classically, and a latent vector of a learned model that captures the image’s details. Experiments show that our method, while currently on par with PNG compression, demonstrates the potential to improve upon today’s compression standards through further model training and tuning.

Keywords: compression · neural networks · image processing.

1 Introduction

Over the past few decades, advancements in networking infrastructure and communication protocols, the development of high-performance storage and cloud infrastructure, and the rapid proliferation of high-dimensional data through user-generated content on the internet have sparked the need for improvements in lossless data compression. Algorithms such as Huffman coding have long achieved optimal expected performance over the potential distributions of the underlying data. However, conditioning on the data enables more efficient compression schemes. For example, [1] demonstrates the potential for a learned non-standard JPEG quantization table to improve image compression performance.

In particular, deep learning advances have demonstrated a profound ability to understand data seen at training time. In particular, auto-encoder machine learning models can learn to encode structured data into a smaller latent space, raising the possibility of using such techniques for compression. Prior work in this area includes [2], which performs image compression by training a model

* These authors contributed equally to this work.

with a loss function that jointly optimizes for two objectives: latent space compression cost and distortion of the reconstructed image (combined using a tuning parameter). However, the use of a machine learning model intrinsically results in lossy compression. To achieve lossless compression, we complement learning based compression with classical algorithms.

Lossless image compression is currently mostly entropy-based, with one of the main ideas being to train a learning based model called the Hyperprior. The Hyperprior learns the distribution of the latent space for better entropy encoding and often relies on arithmetic or Huffman Encoding [3]. One study [4], uses a classical algorithm known as Better Portable Graphics (BPG) for quantization, but the residual at the center of classical BPG is often pure noise and lacks patterns to exploit. Furthermore, this study trains and evaluates the model on a JPEG dataset for lossless compression, where the images would already have much of their latent noise removed (since the images are already lossy-compressed). Our work addresses this problem by employing a K-means nearest color quantization method. We develop a neural model to learn “high frequency” features such as edges and fine details while quantization addresses the background and the main outlines of the image.

Color quantization is a standard technique for trading off image quality for better compression rates under standard compression algorithms such as PNG. The natural question arises of whether we can extend this method to allow for lossless image compression.

The simple way to do this is to compress the exact difference between the quantized image and the original image (which we call the residual) and store it alongside the compressed quantized image. However, the problem is that, in contrast with the quantized image, the residual consists of details that are difficult to compress using standard compression algorithms. Even so, naively compressing the residual alongside the quantized original is known to produce slightly better compression rates. This is promising, but in order to fully make use of the complex redundancies which can be found in the finer details of the residual image, we make use of machine learning.

Therefore, our work uses an autoencoder to store an approximation of the residual in a significantly smaller latent space, thus leaving an approximate quantized image for the standard algorithm to compress.

2 Method

2.1 Overview

Our compression and decompression algorithms rely on both parties^{*} having access to the same auto-encoder model consisting of two parts, the encoder and the decoder. Denote the original image we are looking to compress by I . The first

^{*} We use the term parties in order to make clear what is needed during compression and during decompression. For a more intuitive explanation, we imagine the compression task as a communication problem.

step in the compression algorithm is to compute the quantized image Q (details can be found in section 2.4), consisting of a small number of distinct colors and large, continuous regions of same-colored pixels. As a result, this image is very easy to compress (around 7% of the original image on average).

To ensure lossless compression we take the difference between the original image and the quantized image to obtain the residual $R = I - Q$, which takes values in the range $[-1, 1]$ in each RGB channel, assuming that I and Q take values between 0 and 1 in the 3 RGB channels. Interestingly, we note that compressing R and Q both using classical algorithms (e.g. PNG) already sometimes achieve a slightly better performance. However, the PNG compression of R is barely smaller than original, thus we hypothesize a learning-based method on reconstructing R can significantly improve the compression performance.

Namely, this study uses a convolutional autoencoder to learn the residual R . The purpose of our autoencoder model is to approximate it by compressing it down to a small latent space vector $L = \text{encoder}(R)$. Then, the approximate residual is computed as $\tilde{R} = \text{decoder}(L)$, which also takes values in the range $[-1, 1]$ in each channel. Note that latent L will be part of the tuple that comprises the compressed image; both parties have access to the tuple, thus $\tilde{R}$ (but not to R). Finally, to ensure lossless compression we compute the approximate quantized image $\tilde{Q} = I - \tilde{R}$. This is stored alongside the latent L (using standard lossless compression algorithms) to form the compressed image. Therefore, both parties have access to both $(\tilde{Q}, L)$ and thus also $\tilde{R}$ from L . The decompression algorithm reconstructs the original image exactly as $I = \tilde{Q} + \tilde{R}$. An example of this methodology is portrayed Fig 1. and Fig 2. .

There are some details that require more attention: by construction, the approximate quantized image $\tilde{Q}$ takes values in the range $[-1, 2]$. However, since we expect our model to produce an approximate quantized image $\tilde{Q}$ that is very close to the actual quantized image Q , we can safely assume that the vast majority of values of $\tilde{Q}$ are in the range $[0, 1]$ corresponding to Q . We therefore clip the approximate quantized image to the range $[0, 1]$, obtaining $\tilde{Q}_c$, and store the sparse overflow as a separate image $D = \tilde{Q} - \tilde{Q}_c$. This overflow is very sparse, and therefore we hypothesize to add a negligible overhead for a sufficiently accurate model.

2.2 Compression

The compression algorithm (Algorithm 1) computes all objects we have discussed in the previous section. Note that during compression, the residual R is not only encoded to the latent space, but also decoded, since we have to compute the approximate residual $\tilde{R}$ in order to compute the approximate quantized image $\tilde{Q}$ that we store.

In this section, let us be a bit more careful with the actual format for the computed objects. To achieve lossless compression, we have to work with images of integer values in the range $[0, 255]$ or $[-255, 255]$ by case. Thus, although we convert R to values in the range $[-1, 1]$ when encoding and also get $\tilde{R}$ with values in the same range when decoding, we ultimately use the integer versions both

Algorithm 1 Compression

```

1: procedure COMPRESS( $I$ )
2:    $\tilde{Q} \leftarrow \text{Quantize}(I)$ 
3:    $R \leftarrow I - \tilde{Q}$ 
4:    $L \leftarrow \text{Encoder}((R/255 + 1)/2)$  ▷ Input and output are in range  $[0,1]$ 
5:    $\tilde{R} \leftarrow (\text{Decoder}(L) \cdot 2 - 1) \cdot 255$ 
6:    $\tilde{Q} = I - \tilde{R}$ 
7:    $\tilde{Q}_c = \text{Clip}(\tilde{Q}, [0, 255])$ 
8:    $D \leftarrow \tilde{Q} - \tilde{Q}_c$ 
9:    $l \leftarrow \text{CompressZLib}(L)$ 
10:   $\tilde{q}_c \leftarrow \text{CompressPNG}(\tilde{Q}_c)$ 
11:   $d \leftarrow \text{CompressZLib}(D)$ 
12:  return Pack( $l, \tilde{q}_c, d$ )

```

when storing the compressed versions and when reconstructing the original. Thus, we are not subject to precision errors.

The resulting compressed image is the tuple $(L, \tilde{Q}_c, D)$, which is stored using standard compression algorithms. Namely, we use the zlib library in python to compress L and D since the latent does not exhibit any clear structure and the overflow image is very sparse. For the clipped approximate quantized image $\tilde{Q}_c$, we use PNG.

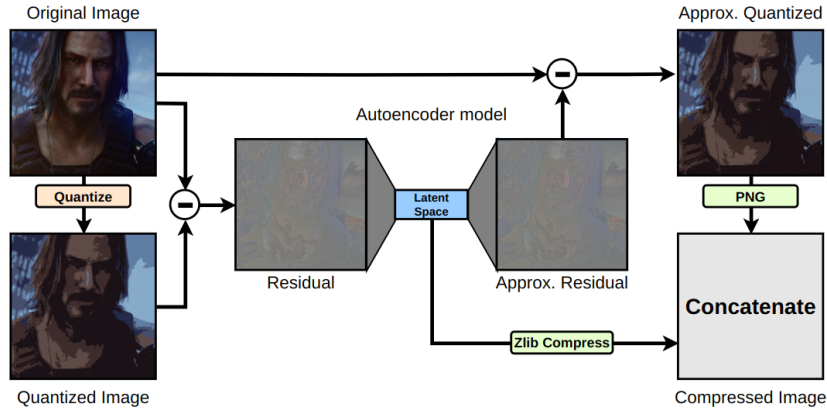


Fig. 1. Compression algorithm visualized. The compression methodology is showed on an example image. In this implementation, PNG and the DELATE algorithm (with zlib implementation) are used for classical algorithms. Note that since residual falls in the range of $[-1, 1]$ per channel, to display the residual we must transform the image to range $[0, 1]$, making the displayed image gray-scaled.

2.3 Decompression

The decompression algorithm (Algorithm 2) uses the same auto-encoder model. Here, however, only the decoder is used. We start by decompressing the tuple $(L, \tilde{Q}_c, D)$ using the standard algorithms we mentioned in the previous section. Then, we pass the latent L through the decoder to get the approximate residual $\tilde{R}$. Finally, we add the approximate residual to the approximate quantized image $\tilde{Q} = \tilde{Q}_c + D$ to get precisely the original image $I = \tilde{R} + \tilde{Q}$. Note that the entire compression scheme is lossless by design. Although the autoencoder is an inherently lossy part of the algorithm, the fact that the approximate quantized image is computed as the exact difference between the approximate residual and the original image means that retrieving the two is sufficient for achieving lossless compression.

Algorithm 2 Decompression

```

1: procedure DECOMPRESS( $l, \tilde{q}_c, d$ )
2:    $L \leftarrow \text{DecompressZLib}(l)$ 
3:    $\tilde{Q}_c \leftarrow \text{DecompressPNG}(\tilde{q}_c)$ 
4:    $D \leftarrow \text{DecompressZLib}(d)$ 
5:    $\tilde{Q} \leftarrow \tilde{Q}_c + D$ 
6:    $\tilde{R} \leftarrow (\text{Decoder}(L) \cdot 2 - 1) \cdot 255$  ▷ Input and output are in range  $[0, 1]$ 
7:    $I \leftarrow \tilde{Q} + \tilde{R}$ 
8: return  $I$ 

```

2.4 Color Quantization

The quantization algorithm we used simply uses the k-Means algorithm to obtain a small number of clusters of similar pixel color. Then, we set the color of each pixel in the quantized image to the closest color from the smaller palette of colors determined by k-Means. Initial experiments involved adding Gaussian blur in the first step to obtain smoother quantized images with larger continuous regions of same-color pixels, as these typically produce better rates for classical compression algorithms. However, this proved to result in blurry contours that worsened the performance of the algorithm on sharp edges and reduced compression rates overall.

For our purposes, we use 7 colors. The resulting quantized images are up to 25 times smaller than the original images when compressed using PNG (Figure 2). Of course, this does not take into account the space occupied by the latent and the fact that the approximate quantized image $\tilde{Q}$ will not be as easy to compress, but the ratio represents a lower bound for the actual compression ratio.

2.5 Architecture and Training

Our autoencoder model architecture works with image patches of size 128×128 . The encoder consists of 4 convolutional layers consisting of 32, 64, 128, and

Algorithm 3 Color Quantization

```

1: procedure COLORQUANTIZE(floatArray[3][Height][Width]  $I$ )
2:    $Q \leftarrow \text{GaussianBlur}(I, \text{kernel\_size} = 9, \sigma = 1.7)$ 
3:    $Q \leftarrow \text{Permute}(Q, \text{order} = (1, 2, 0))$  ▷ shape: [H, W, 3]
4:    $P \leftarrow \text{Reshape}(Q, [-1, 3])$ 
5:    $C \leftarrow \text{SampleRandomRows}(P, \text{num\_colors})$ 
6:   for  $i = 1$  to 10 do
7:      $D \leftarrow \text{PairwiseDistances}(P, C)$  ▷ shape: [N, K]
8:      $L \leftarrow \text{ArgMin}(D, \text{axis} = 1)$  ▷ label for each pixel
9:      $M \leftarrow \text{OneHot}(L, \text{num\_classes} = \text{num\_colors})^\top$ 
10:     $C_{\text{old}} \leftarrow C$ 
11:     $S \leftarrow \text{Sum}(M, \text{axis} = 1)$ 
12:     $C_{\text{new}} \leftarrow (M \cdot P) / (S + \epsilon)$ 
13:     $C \leftarrow \text{Where}(S > 0, C_{\text{new}}, C_{\text{old}})$ 
14:     $D \leftarrow \text{PairwiseDistances}(P, C)$ 
15:     $L \leftarrow \text{ArgMin}(D, \text{axis} = 1)$ 
16:     $Q \leftarrow \text{Reshape}(C[L], \text{shape} = \text{Shape}(Q))$ 
17:     $Q \leftarrow \text{Permute}(Q, \text{order} = (2, 0, 1))$ 
18:     $T \leftarrow I - Q$ 
19:  return  $(T, Q)$ 

```

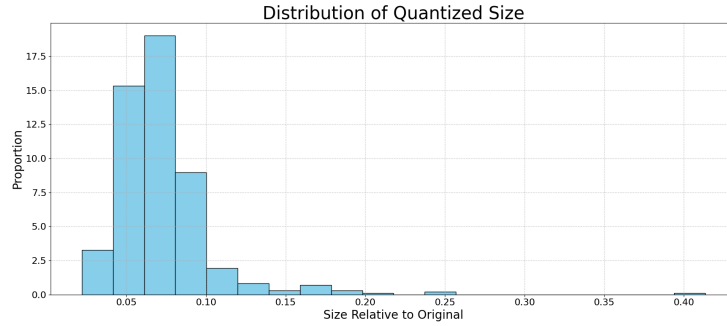


Fig. 2. Distribution of quantized image compressed size as a fraction of the original compressed size for a sample of 500 images. The median ratio is of about 7% of the original compressed size.

256 channels respectively, each followed by group normalization and ReLU activation. After the last of these layers is applied, the resulting latent space has dimension $1024 \times 4 \times 4$, so 8192 in total (compare this to the original size of $3 \times 128 \times 128 = 49152$). The decoder is symmetric, consisting of 4 transposed convolutional layers, each followed by group normalization and ReLU activation. After the last layer of the decoder, we apply the sigmoid activation function. Sigmoid works because we make sure to convert the residual R from the range $[-1, 1]$ to the range $[0, 1]$ before encoding, and then convert the approximate residual $\tilde{R}$ from the range $[0, 1]$ back to the range $[-1, 1]$ when decoding. The total number of parameters is approximately 3702947.

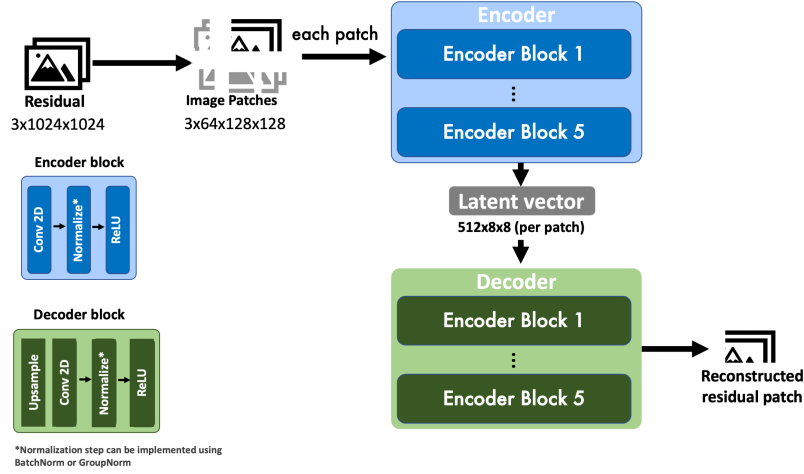


Fig. 3. Autoencoder architecture visualized. Note that an image is partitioned into 128x128 patches to be inputted in the autoencoder.

To use the model on images of full size (1024×1024 in our case), we split the images into patches, pass them through the model, and then reassemble the patches in the original shape. Thus, once the model is trained, it can be used on images of any size, as long as the dimensions are both divisible by 128. The latent space for the full size images is considered to be the concatenation of the latent spaces of all patches. In our case, since $1024/128 = 8$, we have 64 patches per image and thus a latent size of $64 \times 8192 = 524288$ (compare this to the original size of 3145728).

We train the model on a dataset of 5000 PNG images of size 1024×1024 that we assembled. Because there exist few datasets that contain high quality large (1024px) PNG dataset, we assemble our dataset by filtering the Laion2B Hugging Face dataset [5] and Div2k [6], because we noticed that laion2b itself does not have enough high quality PNG images, while div2k only contain 1000

images. From the dataset, PNG images with size larger than 1024×1024 are filtered (images larger than 1024×1024 are cropped in the center). To ensure generalizability of our method and to prevent overfitting, the dataset contain a diverse set of different digital images and photographs. This results in a total of $5000 \times 64 = 320000$ training samples for the model to learn from. We use the L1 loss, which is the absolute value between the residual and the approximate residual outputted by the model. We train the model for 25 epochs on a cluster with 4 nodes, each node consisting of 1 GPU and 4 CPUs. But this is done only for reducing the training speed, as it can also be trained on a single M1 GPU, and can be used with CPU only after training.

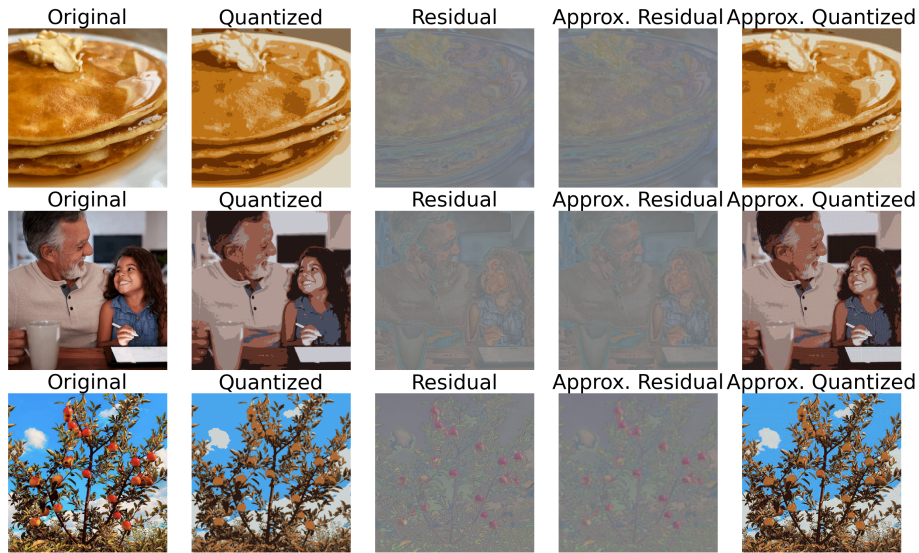


Fig. 4. Results from training the model for 20 epochs on the 5000 image dataset. From left to right, the original image, the color-quantized version, the residual (input to the autoencoder and target in the loss function), the approximate residual outputted by the model, and the approximate quantized image that is left for the standard algorithm to compress.

3 Results

Despite a theoretically grounded methodology and careful implementation, our approach did not yield the expected performance improvements over baseline models. In this section, we present the empirical results and analyze the factors that may have contributed to this outcome.

We start off our experimental by analyzing compression rates for each of the objects we work with. However, we are mostly interested in comparing the size of

the original image compressed by PNG, $\text{CS}[I]$, with the compression size obtained using our algorithm, $\text{CS}[L] + \text{CS}[\tilde{Q}_c] + \text{CS}[D]$. We also add to the comparison the size obtained by compressing Q and R separately using PNG. The results are reported in Table 1.

Looking at the performance on 5 random images from the dataset, we observe that our method performs slightly worse than PNG. As expected, the most difficult object to compress in our tuple $(L, \tilde{Q}_c, D)$ is the approximate quantized image $\tilde{Q}_c$, whose quality is not yet enough for an actual improvement in overall compression size. For example, consider the zoomed-in comparison in Figure 5. Our model learns to encode the residual in a small latent space. When we decode it and construct the approximate quantize, the images look visually identical as a whole, but it turns out that our model is not yet able to produce smooth enough regions for PNG to benefit from.

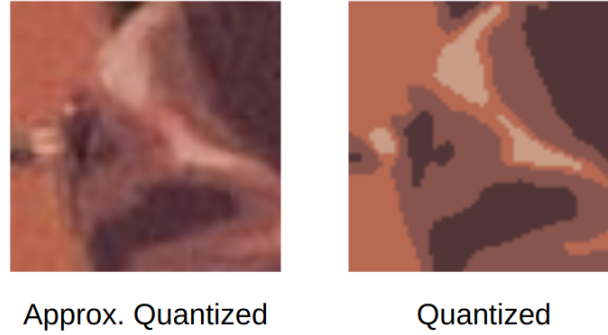


Fig. 5. Zoomed in (64×64 patch) comparison between the approximate quantized image produced by our best model and the actual quantized image.

Fortunately, the quality of the autoencoder model directly influences the compression size of $\tilde{Q}_c$, which means that, improving the model reduces compression size, with the only theoretical lower bound to the total compression size being $\text{CS}[L] + \text{CS}[Q]$. In contrast, note that the compressed latent size $\text{CS}[L]$ is not influenced by the image being compressed, and should rather be seen as a hyperparameter. Increasing the latent space will result in worse compression rates for L and better compression rates for $\tilde{Q}_c$, thus introducing a tradeoff. In our study, we have chosen the latent space that performed best given our architecture. Finally, observe that the overflow difference D that resulted from clipping $\tilde{Q}$ to the range $[0, 255]$ is negligible.

Table 1. Comparison of compressed file sizes in bytes. We report the size of the original PNG image, the size of the file stored using our method, and the sizes of the objects we use in our compression algorithm. We report these metrics for the first 5 images in our dataset. For a more complete analysis, see Figure 6.

Image	PNG	Our Method	L	$\tilde{Q}_c$	D
1	224925	1152155	7443	795928	348784
2	1242049	1442513	7369	1397244	37900
3	697722	945229	7455	931346	6428
4	1706613	1855593	7671	1727375	120547
5	945452	1194771	7755	1179321	7695

For a more careful analysis of the compression rates achieved by our method, we plot the distribution of the compression ratio between the file produced by our algorithm and the original image stored using PNG (Figure 6).

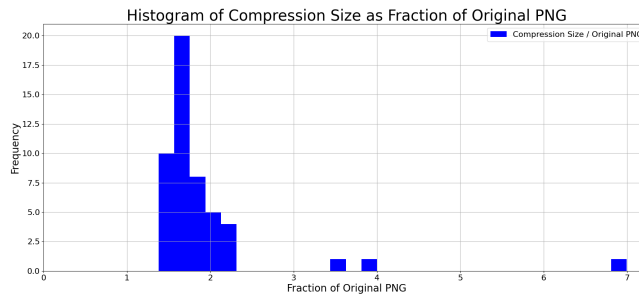


Fig. 6. Distribution of compressed image sizes using our method as a fraction of the original image compressed size using PNG.

4 Challenges and Experiments

We report on the different iterations we went through while training the best model we currently have available. Then, we discuss the attempts at improving this model, and the reason they failed. Finally, we propose future directions we have not yet tried, and explain why we think they could potentially improve the model.

4.1 Encoding Full Image

Early on in our research we experimented with an autoencoder architecture operating on the full sized 1024×1024 images. Although such an architecture

could be somewhat motivated in scenarios where non-local redundancies exist (e.g. an image of a repeating pattern), we have reached the conclusion that learning on the full image suffers major flaws.

While convolutional layers are able to shrink the dimension of the image by a factor of 2 at each step, even after applying 4 such layers, we would still be left with a dimension of 64×64 . Thus, distant parts of the image would have never interacted throughout the encoding and decoding process anyways. One solution we have tried to implement was to insert two fully-connected layers to and from the flattened latent space of desired size. Both with and without the fully-connected layers, training the model was very computational intensive, and it also produced very poor results.

Thus, we have ultimately switched to the patch-based model, which is easier to train, takes vastly less memory to store, and produces visually perfect results.

4.2 Different Loss Functions

Over the different model iterations we tried, we have experimented with multiple loss functions. The standard L2 Loss produced decent results, and it was the starting point in our research. Later on, we have experimented with L1 Loss, both alone and combined with other relevant loss functions. The first one is Total Variation Loss, also known as Color Similarity Loss) which encourages more crisp edges and less noisy artifacts. The second one is Edge Loss, which is motivated by the fact that we observed the biggest differences between R and $\tilde{R}$ along edges of objects in the original image. With L1 and color similarity loss and edge loss, the model achieved slightly better performance in resulting compression size than L2 (with the two L1 variants having similar performance). Thus we proceed to use L1 with edge loss in the rest of the experiments.

4.3 Recursive Compression

Another idea we have tried is the following: Suppose we use the compression algorithm on an image I and obtain the tuple $(L, \tilde{Q}_c, D)$. At this point, the most significant part of our compressed image file is $\tilde{Q}_c$, so it make make sense to try compressing it again: We treat $\tilde{Q}_c$ as a new image and pass it through the compression algorithm, obtaining $(L', \tilde{Q}'_c, D')$. Then, we can compress our original image I as a tuple $(L, L', \tilde{Q}'_c, D', D)$ (note that $\tilde{Q}_c$ is no longer needed). The decompression algorithm reconstructs the original image as $I = \text{decoder}(L) + \text{decoder}(L') + \tilde{Q}'_c + D' + D$. Thus, this recursive compression scheme is an improvement on the default scheme if and only if $\text{CS}[\tilde{Q}_c] - \text{CS}[\tilde{Q}'_c] > \text{CS}[L'] + \text{CS}[D']$ (where CS denotes compressed size under some standard compression algorithm).

In practice, even though the quantized images Q seem to become simpler after each recursive step, it turns out that compression rates for the approximate quantized image $\tilde{Q}$ increase. This makes sense in the current situation where the model produces approximate quantized images that are harder to compress than the original image. However, for a model that is able to produce images that are

easier to compress, this recursive technique might be worthy of consideration. Essentially, it introduces a simpler trade-off between latent size and approximate quantized image compression rate (since we have one latent space per recursive step). Ultimately however, retraining the model with the desired latent size is preferable.

4.4 Overlapping Patches

One issue with the model operating on patches is that after reassembling the outputs, the resulting image tends to suffer from grid-like artifacts because of the poor quality near the patch edges. We tried to fix this using multiple strategies involving patch overlapping. The first one consists of splitting the image into patches two times, shifting the patch grid by an offset the second time. Then, we pass both patch grids through the autoencoder and reassemble the input by taking the mean of the two reassembled outputs. This has improved compression rates and we decided to use it all our subsequent models.

The second strategy consists of lowering the stride when splitting into patches and reassembling. This way, the reassembled image would actually consist of patches of smaller size (e.g. 112×112). These are obtained by encoding and decoding overlapping patches of the original size 128×128 but only using their interior. However, this method amplifies the grid-like artifacts, which results in worse compression rates.

4.5 Different End-Compression Algorithms

We managed to get much better compression rates by using alternative compression algorithms for the approximate quantized image $\tilde{Q}_c$, such as WebP, and PNG with adaptive color palette. However, this meant that, in order to provide a fair comparison with the benchmark, we also had to convert the original images to the new format. In both cases, the original image still experienced better compression rate than ours. (This also includes Lossless JPEG Compression, in which around 200 KiB are saved, yet the original also become smaller and still experienced better compression rate overall).

A more promising direction consists of using the fact that, although the approximate quantized image $\tilde{Q}$ does not yet consist of continuous regions of same-colored pixels, it is still relatively smooth. Thus, we can try to decompose it into noise of small amplitude and a smooth image. We implemented this idea by quantizing $\tilde{Q}$ (resulting in a quantized image possibly different from $\tilde{Q}$) and storing the difference $\tilde{Q} - \text{quantize}(\tilde{Q})$ separately. However, this approach did not improve performance, as the noise obtained by subtracting the quantized version turned out to have high amplitude for many pixels, resulting in undesirable compression rates.

However, most of the high-amplitude pixels are located along the edges of objects in the original image (Figure 7), so we have also tried a compression scheme where we separate the low bits and the high bits of the values in the difference and use two different compression algorithms for the two. For the low

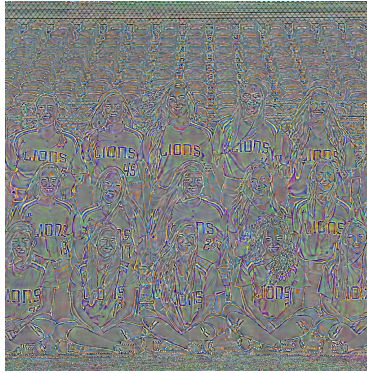


Fig. 7. Sample of the difference $\tilde{Q} - \text{quantize}(\tilde{Q})$ exaggerated 5 times in order to see the high amplitudes observed along the edges of objects.

bits, which we expect to consist of pure noise, we just store the values using zlib. For the high bits, we use PNG, since we expect a very sparse image where only the edge pixels are non-zero.

In practice however, we observe that the compression rate gets increasingly worse when we increase the number of low bits encoded using zlib. Therefore, we ultimately did not end up using the idea.

4.6 Denoising the Approximate Residual

One idea that led to better compression rates was to denoise the approximate residual $\tilde{R}$ produced by the autoencoder model using the cv2 python library. Note that we can apply any transformation on this approximate residual without modifying any other part of the algorithm. This is because our compression scheme does not make any assumptions on $\tilde{R}$ except that it must take values in the range $[-1, 1]$ (before converting to $[-255, 255]$). Therefore, denoising this residual eliminates a component that clearly does not represent learned details in the image. However, since the original image can also contain noise, which should be encoded by our autoencoder, this strategy is a two edged sword. In practice, denoising the approximate residual slightly improved compression rates, which strengthens the idea that our compression scheme could currently benefit from an improved model.

4.7 Denoising the Approximate Residual by Storing Color Palette

We can also denoise the approximate residual by storing some information about the original image. For example, storing the color palette P of the original image and the exact quantized image Q allows us to apply the following transformation to the approximate residual $\tilde{R}$ at compression and decompression. Let $I^* = \text{quantize}(\tilde{R} + Q, \text{palette} = P)$. Then, we can define $R^* = I^* - Q$ and $Q^* =$

$I - R^*$. Thus, we compress the image as (L, P, Q, Q^*) and are able to losslessly reconstruct the original image in the following way. First, get $\tilde{R} = \text{decode}(L)$. Then, $I^* = \text{quantize}(\tilde{R} + Q, \text{palette} = P)$. Finally, $R^* = I^* - Q$ and $I = R^* + Q^*$. The motivation for performing this complicated transformation is that correcting the color of $I^* = \tilde{R} + Q$ to match that in the original image I should improve the quality of $\tilde{Q}$ for a small overhead of storing P and Q . In practice, the resulting Q^* is in fact about 50 kB smaller than $\tilde{Q}$ under PNG. However, storing the full color palette turns out to require at least 100 kB (and Q is negligible). Storing a smaller palette size could potentially lead to an overall improvement in some cases, but we gave up on the idea since the 50 kB reduction is very small and some images require even larger palettes of sizes up to 1 MB, which makes the idea highly improbable to work.

4.8 Top-K

The most effective method we have used in order to denoise the output of the autoencoder model is Top-k. This refers to keeping only a fraction of the weights fitted by the model during training and setting all other weights to 0. The motivation behind this approach is to enforce sparsity in the learned representation. By retaining only the top k of the largest weights (by magnitude), we reduce the influence of small, potentially noisy activations that may not contribute meaningfully to the reconstruction. This helps the autoencoder focus on the most salient features of the input data, which often correspond to the underlying structure rather than to noise. Our final algorithm employs Top-k.

5 Conclusion

Our study proposes a general framework for incorporating any standard image compression algorithm into an autoencoder-based compression scheme in order to achieve lossless compression. The compression rates produced using our framework benefit directly from the improvements made to the autoencoder model. The experiments we performed using the different iterations of our model indicate that there is still a lot of room for improvement on architecture and training, and these improvements actually impact the compression rate. Therefore, we believe that at this point, the most promising avenue of future research based on framework focuses on the architecture and training.

The other component that could possibly benefit from a new approach is the end compression algorithm. We have chosen PNG for this study because it is standard, versatile, and provides us a direct comparison with PNG on our dataset. However, it might be beneficial to shift the focus on better compression algorithms for the approximate quantized image produced using by the autoencoder model. So far we have tried many standard compression methods, but perhaps the best way to move forward is entropy coding. We already observed that the difference between our approximate quantized image $\tilde{Q}$ and the true quantized image Q has structure determined by the way our autoencoder was trained (the most obvious

examples being grid-like effects and the artifacts along the edges of objects). Therefore, finding a way to make use of this structure more efficiently could be the breakthrough in our method (for example, other studies learn to predict a probability distribution of each pixel value based on previous pixel values for entropy coding).

In conclusion, we believe that our compression scheme could represent a versatile framework for incorporating any standard algorithm into an learning-based compression, but future research should prioritize improving both the autoencoder architecture and the final compression stage. Leveraging the structured artifacts in the quantized image through advanced entropy coding techniques may lead to significant gains in compression performance in future work.

References

- [1] M. Hopkins, M. Mitzenmacher, and S. Wagner-Carena, *Simulated annealing for jpeg quantization*, 2017. arXiv: [1709.00649](#) [[cs.MM](#)]. [Online]. Available: <https://arxiv.org/abs/1709.00649>.
- [2] L. Theis, W. Shi, A. Cunningham, and F. Huszár, *Lossy image compression with compressive autoencoders*, 2017. arXiv: [1703.00395](#) [[stat.ML](#)]. [Online]. Available: <https://arxiv.org/abs/1703.00395>.
- [3] D. A. Huffman, “A method for the construction of minimum-redundancy codes,” *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952. DOI: [10.1109/JRPROC.1952.273898](#).
- [4] F. Mentzer, L. V. Gool, and M. Tschannen, *Learning better lossless compression using lossy compression*, 2020. arXiv: [2003.10184](#) [[cs.CV](#)]. [Online]. Available: <https://arxiv.org/abs/2003.10184>.
- [5] *Opendiffusionai/laion2b-squareish-1024px · Datasets at Hugging Face*, Mar. 2025. [Online]. Available: <https://huggingface.co/datasets/opendiffusionai/laion2b-squareish-1024px> (visited on 05/10/2025).
- [6] E. Agustsson and R. Timofte, “Ntire 2017 challenge on single image super-resolution: Dataset and study,” pp. 1122–1131, 2017. DOI: [10.1109/CVPRW.2017.150](#).